

claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-ab76c8589a16bfea3.jsonl`  
- SHA-256: `696efe4972fad9dfcc228cab24dd1f589dfffb4142f093d3db7b6bbafe69935eb`  
- Source modified: `2026-07-08T06:17:53+00:00`  
- Imported at: `2026-07-08T15:59:28+00:00`  
- project: `wf_5f13fc79-bad`  
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`
```

Transcript

1. user (2026-07-08T06:14:14.452Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).

The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):

- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config + source uri) and needs NO control-plane DB. It reuses orchestration.__stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing, never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #1.

CLAIM (medium/test-coverage) backend/worker/___main__.py:865

summary: The dispatcher->worker contract (compile argv flags and the staged spec JSON schema) is never round-tripped end-to-end; each side is tested against its own hand-built fixture, so a flag/key drift between them passes CI but breaks prod.

failure scenario: CloudRunCompute.run_compile (cloud_run.py:323) emits argv ["compile", "--out-uri", ..., "--spec-uri", ..., "--done-uri", ..., "--set", ...], and _dispatch_compile_remote stages a spec with keys {video_id, video_uri, output_name, locale, time_pairs, stitching} (orchestration.py:1330-1348); cmd_compile parses that argv via build_parser()'s compile subparser (worker/___main__.py:865) and reads those exact spec keys (worker/___main__.py:490-496). No test connects the two halves: test_run_compile_dispatches_and_returns_marker asserts run_compile's emitted argv via a fake client that never calls build_parser(), and test_parser_compile_accepts_spec_and_done_uris parses a HAND-WRITTEN argv; likewise the spec is written by test fixtures independent of the CP producer. If a flag is renamed/removed or made required on only one side (e.g. --spec-uri), or a spec key drifts (video_uri vs source_uri, time_pairs), both tests still pass, but the real worker exits 2 on an unrecognized/missing-required arg (no marker -> opaque failed job via crash-fast) or KeyErrors reading the spec. The duplicated string literals across producer and consumer make this drift easy to introduce and currently unguarded.

2. assistant (2026-07-08T06:14:32.096Z)

I'll investigate this claim by reading the actual code and tests. Since the paths reference "undefined", I'll work from the actual repo at the working directory. Let me first pull and locate the relevant files.

3. user (2026-07-08T06:14:33.967Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload (#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
---BRANCH---
codex/issue-523-sd-reel-plan

4. user (2026-07-08T06:14:35.021Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521)
a5d6246 serving: re-queue interrupted compile jobs on restart (#329 reliability) (#460)
b09a1d4 serving: re-queue interrupted compile jobs on restart (#329 reliability)
d5a1fea fix: Phase 2 ? Q7 (motion type-clean) + B2 (Gemini error-flagging) + D4 (deep config merge) (#373)
947a36b docs: update CODE_CLEANUP_AUDIT ? Phase 1 complete, Phase 2 prioritized (#372)
739d1f4 fix: Q2 motion return type + D11 compiler codec docstring (#362)
bab72c6 docs: update cleanup project tracker ? Phase 1 items shipped (#357)
8e3aff1 docs: 2026-06-24 code-cleanup audit ? 33-item project doc (Phase 1 approval gate) (#352)
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329) (#339)
e221cb9 docs(data): GCS corpus source-of-truth map + Phase-1 metadata migration (#316)
e6110ce feat(api): GET /api/health + return download_url from /api/compile (M1/P6) (#268)
dd30e9b feat(workspace): per-video workspace convention ? design doc + Phase 0 helper (#264)
8092a39 refactor(segmenter): decompose yolo_hybrid process_video into phase helpers (#227)
428ee44 address follow-up review on #169: correct factual error in item-3 investigation note (fusion_compare.py, fusion_golden.py, ablation.py + tests *do* exist per ls/grep on the tree; person features in fusion_hybrid + detectors/fusion_features.py; remaining M0 in-container task is wiring into compare). Cleaned duplicate sequencing paragraph. Verified files present. Plan now accurate. M0 item 3 placement remains correct and even stronger.
318f6c2 test: positive tests for Item 2 (config) - shallow section replacement, unknown keys

warning, get shim, roundtrip + living doc update for T2 post #157
8439c24 feat(personalization): per-user store (v4/v5) + serving bridge groundwork (Phase 1, additive) (#150)
e75ec35 feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails + locked structural-guard decision (#145)
8836dfe chore: scripts/ for run-drivers + CODE_MAP \$0 file-placement guidelines (#138)
3461898 feat(api): enforce stitching.min_shot_count on /api/compile (#107)
30e4899 fix(api): thread config stitching paddings into live VideoCompiler sites
5eaba74 feat(api): chunked upload + highlight download (B2+B3) - static UI compile fix + download link (#99)
595a5a1 ci: add ruff lint lane + fix all violations; remove never-compiled flyer stub (#92)
05bf49e docs: field-research recruiter flyer (C13 interviews + Phase-2 recording rules) (#86)
95c118c test: close the worst coverage gaps (API seam, compiler, live WasbRunner) (#70)
1358521 review: address #42 nits ? dynamic section scan, shim comment, scalar-default test
6668e02 Merge pull request #18 from avidullu/docs/phase1-video-spec
25086c8 docs: Phase-1 golden-set video spec; capture values; train/eval guardrail
6fe3612 Merge: Phase 3 eval driver, Phase 4 offline-segmenter foundations, roadmap/ADRs

5. assistant (2026-07-08T06:14:37.350Z)

The relevant commit is `dac6736`. Now let me read the actual files involved in the claim.

6. user (2026-07-08T06:14:40.802Z)

```

977 backend/worker/__main__.py
516 backend/compute/cloud_run.py
1514 backend/orchestration.py
59 backend/config/placement.py
702 backend/pipeline/stitcher/compiler.py
3768 total

```

7. user (2026-07-08T06:14:41.840Z)

```

commit dac6736cb23a681135b707d768014820b239a981
Author: Avi <avi.dullu@gmail.com>
Date:   Wed Jul 8 11:29:33 2026 +0530

```

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, \$9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: deployment.phase_placement {validation|segment|compile} ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches `worker compile` on the L4, bucket-polls a done-marker, and returns the same {status, filepath, download_url} shape the SPA expects. The worker stitches via the SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
- NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
- validation delegation: phase_placement.validation moves the validator suite off the

control-plane via the existing --skip-validation lever (#513), guarded so an in-process run always validates.

- cloud-serving preset places compile on the L4.

Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).

Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to test_compute_backend.py (run_compile), test_worker.py (cmd_compile), test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

backend/compute/__init__.py	2 +
backend/compute/base.py	34 +++
backend/compute/cloud_run.py	100 +++++-
backend/config/__init__.py	16 ++
backend/config/models.py	52 ++++
backend/config/placement.py	59 ++++
backend/config/presets.py	10 +-
backend/orchestration.py	322 ++++++-----
backend/pipeline/stitcher/compiler.py	94 +++++-
backend/worker/__main__.py	139 +++++++
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md	169 +++++++
docs/COMPUTE_DECOUPLED_SERVING/README.md	1 +
tests/test_api_cloud_dispatch.py	70 +++++
tests/test_compile_dispatch.py	204 +++++++
tests/test_compiler.py	64 +++++
tests/test_compute_backend.py	111 ++++++
tests/test_config_deployment.py	5 +
tests/test_phase_placement.py	138 +++++++
tests/test_worker.py	175 +++++++

19 files changed, 1710 insertions(+), 55 deletions(-)

8. assistant (2026-07-08T06:14:45.932Z)

Now let me read the key code sections. Let me look at the worker's compile subcommand, the cloud_run dispatcher, and the orchestration spec staging.

9. user (2026-07-08T06:14:47.890Z)

```
440         returncode: int,
441         storage_cfg: dict,
442         scratch: str,
443     ) -> None:
444         """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
445         dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
446         try:
447             marker = {
448                 "video_id": video_id,
449                 "returncode": returncode,
450                 "status": result.get("status", "failed"),
451                 "download_url": result.get("download_url", ""),
452                 "reel_uri": result.get("reel_uri", ""),
453                 "message": result.get("message", ""),
454             }
455             local = os.path.join(scratch, "_compile_done.json")
456             with open(local, "w", encoding="utf-8") as f:
457                 json.dump(marker, f)
458             storage.put(local, done_uri, config=storage_cfg)
459             logger.info("[worker] wrote compile marker -> %s", done_uri)
460         except Exception as e: # never fail a good stitch because the marker push hiccuped
461             logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)
```

```

462
463
464     def cmd_compile(args: argparse.Namespace) -> int:
465         """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
466         stitch is the
467         2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
468         4K reel). Read
469         the control-plane-staged compile-spec (resolved time-pairs + stitching config +
470         source URI), stitch
471         the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
472         there is no
473         forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
474         the dispatcher's
475         bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
476         requires."""
477         from backend import orchestration
478         from backend.config import StitchingConfig
479
480         config = load_config(args.config) if args.config else load_config()
481         _apply_overrides(config, args.set)
482         storage_cfg = config.storage.model_dump()
483
484         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
485         os.makedirs(scratch, exist_ok=True)
486         # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
487         the bucket.
488         config.output.output_dir = scratch
489         # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
490         value as a
491         # fallback so reel_object_uri (which reads storage.output_root) always resolves the
492         reel's bucket
493         # destination even if the override were dropped.
494         if not (config.storage.output_root or "").strip():
495             config.storage.output_root = args.out_uri
496
497         spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
498         video_id = str(spec.get("video_id") or "")
499         out_name = str(spec.get("output_name") or "")
500         source_ref = str(spec.get("video_uri") or "")
501         locale = spec.get("locale")
502         time_pairs = [
503             (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
504         ]
505         stitching = StitchingConfig(**(spec.get("stitching") or {}))
506
507         print(
508             f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
509             f"encoder={stitching.encoder} out={out_name}"
510         )
511
512         result = orchestration._stitch_reel(
513             config,
514             stitching,
515             video_id,
516             source_ref,
517             time_pairs,
518             out_name,
519             locale,
520             lambda stage: print(f"[worker] compile: {stage}"),
521         )
522         status = str(result.get("status") or "failed")
523         rc = 0 if status == "success" else 3
524         if status != "success":
525             print(f"[worker] compile FAILED: {result.get('message')}")
526

```

```

518         # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
519         if getattr(args, "done_uri", None):
520             _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
521         return rc
522
523
524     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
525     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
526     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
527
528     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
529     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
530     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
531     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
532
533
534     def _label_clip_name(fname: str) -> str:
535         """Derive the clip name (the `videos/<name>.<ext>` join key) from a label
filename.
536
537         Strips both the `.rallies.csv` / `.csv` suffix and an optional leading ISO-date
prefix, so a
538         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
539
540         * `2026-06-22_GX020094.rallies.csv` -> `GX020094` (canonical, date-
stamped)
541         * `GX020094.rallies.csv` -> `GX020094` (legacy bare ? same
stem)
542         * `GX020094_proxy.rallies.csv` -> `GX020094_proxy` (`_proxy` is part of
the name, kept)
543         * `2026-06-21_mahadevpura_1.rallies.csv` -> `mahadevpura_1` (underscores in
<name> survive)
544
545         Without the date strip, `--trajectory-source detector` can't match a dated label
to its bare
546         video stem, so every clip is skipped and train aborts with `<2 videos`
(DATA_IN_GCS §7b #1).
547         """
548         name = fname.replace(".rallies.csv", "").replace(".csv", "")
549         return _LABEL_DATE_PREFIX.sub("", name)
550
551
552     def _probe_video_fps_width(path: str):
553         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
554
555         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
556         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
557         cv2's `_probe_fps_width` silently defaults to (30, 1280) on a read miss ? fine for
the
558         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
559         probe with ffprobe (same tool as `get_video_duration`) and return None so the
caller SKIPS

```

10. user (2026-07-08T06:14:48.876Z)

```

820         sub = p.add_subparsers(dest="cmd", required=True)
821

```

```

822 pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
823 pi.add_argument(
824     "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
825 )
826 pi.add_argument(
827     "--out-uri",
828     required=True,
829     help="output prefix (outputs/<video_id>/? is appended)",
830 )
831 pi.add_argument(
832     "--segmenter", default=None, help="default: indexing.default_segmenter"
833 )
834 pi.add_argument(
835     "--config", default=None, help="config.json path (default: ./config.json)"
836 )
837 pi.add_argument(
838     "--scratch", default=None, help="local scratch dir (default: a temp dir)"
839 )
840 pi.add_argument("--max-frames", type=int, default=None)
841 pi.add_argument(
842     "--done-uri",
843     default=None,
844     help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
845     "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
846     "Default-OFF: omit it for the normal local run.",
847 )
848 pi.add_argument(
849     "--set",
850     action="append",
851     default=[],
852     metavar="k=v",
853     help="config override, e.g. --set indexing.skip_ai_handoff=true",
854 )
855 pi.add_argument(
856     "--skip-validation",
857     action="store_true",
858     help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
859     "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
860     "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
861     "`worker infer` CLI still validates.",
862 )
863 pi.set_defaults(func=cmd_infer)
864
865 pc = sub.add_parser(
866     "compile",
867     help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
868 )
869 pc.add_argument(
870     "--spec-uri",
871     required=True,
872     help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
873     "config + source uri + video_id)",
874 )
875 pc.add_argument(
876     "--out-uri",
877     default="",
878     help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
879     "Also forwarded via --set storage.output_root; either resolves the reel path.",
880 )

```

```

881     pc.add_argument(
882         "--done-uri",
883         default=None,
884         help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
885         "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
886     )
887     pc.add_argument(
888         "--config", default=None, help="config.json path (default: ./config.json)"
889     )
890     pc.add_argument(
891         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
892     )
893     pc.add_argument(
894         "--set",
895         action="append",
896         default=[],
897         metavar="k=v",
898         help="config override, e.g. --set storage.output_root=gs://bucket",
899     )
900     pc.set_defaults(func=cmd_compile)
901
902     pt = sub.add_parser(
903         "train",
904         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
905     )
906     pt.add_argument(
907         "--corpus-uri",
908         required=True,
909         help="prefix containing labels/ (+ videos/ for "
910         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
911     )
912     pt.add_argument(
913         "--out-uri",
914         required=True,
915         help="output prefix (weights/<version>/ is appended)",
916     )
917     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
918     pt.add_argument(
919         "--trajectory-source",
920         choices=["synth", "detector"],
921         default="synth",
922         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
923         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
924         "(needs videos/ in the corpus; honours detector_impl).",
925     )
926     pt.add_argument(
927         "--segmenter",
928         default="wasb_hybrid",
929         help="trajectory hybrid whose detector is used for --trajectory-source
detector",
930     )
931     pt.add_argument(
932         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
933     )
934     pt.add_argument("--config", default=None)
935     pt.add_argument("--scratch", default=None)
936     pt.add_argument(
937         "--fps",
938         type=float,
939         default=30.0,
940         help="synth trajectory fps (detector probes the video)",
941     )
942     pt.add_argument(

```



```

943         "--frame-width",
944         type=float,
945         default=1280.0,
946         help="synth trajectory frame width (detector probes the video)",
947     )
948     pt.add_argument("--set", action="append", default=[], metavar="k=v")
949     pt.set_defaults(func=cmd_train)
950     return p
951
952
953     def _setup_logging(verbose: bool) -> None:
954         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
955         diagnostics are
956         VISIBLE ? the worker previously ran with no logging config, so every
957         segmenter/detector log
958         (detector choice, 'Running native WASB inference', availability failures) was
959         silently dropped.
960         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
961         (the CLI
962         entry), not at import, so importing the module never reconfigures a caller's
963         logging."""
964         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
965         library
966         # configured logging at import), which would leave the level at WARNING and re-
967         suppress the
968         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
969         verbosity.
970         logging.basicConfig(
971             level=logging.DEBUG if verbose else logging.INFO,
972             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
973             datefmt="%H:%M:%S",
974             force=True,
975         )
976
977     def main(argv=None) -> int:
978         args = build_parser().parse_args(argv)
979         _setup_logging(getattr(args, "verbose", False))
980         return args.func(args)
981
982     if __name__ == "__main__":

```

11. assistant (2026-07-08T06:14:51.596Z)

Now the cloud_run dispatcher and the orchestration staging.

12. user (2026-07-08T06:14:53.442Z)

```

290         "video_id) ? treating as failure",
291         self._job_name,
292     )
293     rc = rc or 1
294     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
295     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
296     LOG.info(
297         "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
298     )
299     return ComputeResult(
300         returncode=rc,
301         outputs_uri=outputs_uri,
302         video_id=video_id,
303         report=report,
304         execution=str(execution or ""),
305     )

```

```

306
307     def run_compile(
308         self,
309         spec: "CompileJobSpec",
310         *,
311         on_wait: "Callable[[float], None] | None" = None,
312     ) -> ComputeResult:
313         """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on
the L4 (NVENC),
314         bucket-poll the done-marker, and return the worker's rc + marker. Reuses the
EXACT
315         dispatch?poll?marker machinery as ``run_infer`` (``await_marker`` ? crash-fast
+ timeout
316         rescue), so a stitch that crashes or overruns fails the same way a detection
does ? not a
317         ~65-min opaque hang."""
318         out_root = spec.out_uri.rstrip("/")
319         token = self._token or uuid.uuid4().hex
320         # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for
the same
321         # bucket can never collide on a marker.
322         done_uri = f"{out_root}/_compile/{token}.done"
323         argv: List[str] = [
324             "compile",
325             "--out-uri",
326             spec.out_uri,
327             "--spec-uri",
328             spec.spec_uri,
329             "--done-uri",
330             done_uri,
331         ]
332         # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
333         # never re-dispatches, so the only overrides are the CP-forwarded
storage/encoder knobs.
334         for kv in spec.config_overrides:
335             argv += ["--set", kv]
336
337         execution = self._client.run_job(
338             self._job_name, argv, region=self._region, project=self._project
339         )
340         LOG.info(
341             "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",
342             self._job_name,
343             execution,
344             done_uri,
345         )
346         marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
compile")
347         video_id = str(marker.get("video_id", "") or spec.video_id)
348         rc = int(marker.get("returncode", 1))
349         LOG.info(
350             "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
351             self._job_name,
352             video_id,
353             rc,
354         )
355         # The marker carries the worker's stitch result (download_url / reel_uri /
status); surface it
356         # as ``report`` so the dispatcher returns the same {status, filepath,
download_url} shape the
357         # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
358         return ComputeResult(
359             returncode=rc,
360             outputs_uri=str(marker.get("reel_uri", "") or ""),

```

```

361         video_id=video_id,
362         report=marker,
363         execution=str(execution or ""),
364     )
365
366     def _await_marker(
367         self,
368         done_uri: str,
369         execution: object,
370         on_wait: "Callable[[float], None] | None",
371         *,
372         label: str,
373     ) -> dict:
374         """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer`` and
375         ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting tick
reaches the
376         caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative
path in
377         ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the
dispatch handler
378         maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-
cancel)."""
379         try:
380             return poll_until(
381                 lambda: self._poll_check(done_uri, str(execution or "")),
382                 self._policy,
383                 label=label,
384                 logger=LOG,
385                 on_tick=on_wait,
386             )
387         except PolicyTimeout as timeout_exc:
388             return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
389
390     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
391         """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
392
393         The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
394         wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
395         FAILURE* marker resolves through the normal path with its real returncode. Only
when the
396         marker is absent do we consult the execution status: a worker that TERMINATED
without ever
397         writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
398         otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
399         opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
400         failure.
401
402         Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
403         ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
404         will ever appear)."""
405         marker = self._read_marker(done_uri)
406         if marker is not None:
407             return marker # success signal ? authoritative even if status later flips
408         state = self._terminal_execution_state(execution)
409         if state is None:

```

```

1290         notify("uploading reel")
1291         # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1292         # project/client) is used, not a fresh default.
1293         storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1294         except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1295         logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1296
1297         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1298         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1299         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1300         notify("finished")
1301         return {
1302             "status": "success",
1303             "filepath": out_path,
1304             "download_url": download_url,
1305             "video_id": video_id,
1306         <<<<<<< Updated upstream
1307             "reel_uri": reel_uri or "",
1308         =====
1309             "filename": out_name,
1310             "rendition": rendition,
1311         >>>>>>> Stashed changes
1312         }
1313
1314
1315     def _dispatch_compile_remote(
1316         cfg,
1317         db,
1318         video_id: str,
1319         segment_ids: List[int],
1320         output_filename: str,
1321         locale: Optional[str],
1322         notify,
1323     ) -> Dict[str, Any]:
1324         """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
1325         control-plane owns the DB, so it RESOLVES the reel's segments here
(`_resolve_compile`), stages a
1326         small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
1327         to the bucket, dispatches a `_compile` Cloud Run Job on the L4 (NVENC), bucket-
polls the
1328         done-marker, and returns the same `{status, filepath, download_url}` shape the SPA
expects. The
1329         worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
1330         `_maybe_dispatch_remote` (the DETECT dispatch)."""
1331         from backend.compute import CompileJobSpec, get_compute_backend
1332
1333         notify("resolving")
1334         try:
1335             video, kept, out_name = _resolve_compile(
1336                 db, cfg, video_id, segment_ids, output_filename
1337             )
1338         except _CompileError as e:
1339             return {"status": "failed", "message": e.detail, "video_id": video_id}
1340
1341         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1342         storage_cfg = getattr(cfg, "storage", None)
1343         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1344         compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
1345         if compute is None:

```

```

1346         # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
1347         # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ?
a slow reel
1348         # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
1349         logger.warning(
1350             "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
1351             "stitching in-process on the control-plane instead.",
1352             out_root,
1353         )
1354         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1355         return _stitch_reel(
1356             cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
1357         )
1358
1359         # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe
on the wire);
1360         # the stitching config is the control-plane's RESOLVED one, with the encoder swapped
to the L4's
1361         # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes
the reel.
1362         stitch_dump = stitching.model_dump()
1363         stitch_dump["encoder"] = getattr(
1364             getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
1365         )
1366         spec_payload = {
1367             "video_id": video_id,
1368             "video_uri": video["filepath"],
1369             "output_name": out_name,
1370             "locale": locale,
1371             "time_pairs": [
1372                 [float(s["start_time"]), float(s["end_time"])] for s in kept
1373             ],
1374             "stitching": stitch_dump,
1375         }
1376         token = uuid.uuid4().hex
1377         spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
1378         try:
1379             notify("staging compile spec")
1380             with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
1381                 local_spec = os.path.join(td, "spec.json")
1382                 with open(local_spec, "w", encoding="utf-8") as f:
1383                     json.dump(spec_payload, f)
1384                 storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
1385         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
1386             logger.error(
1387                 "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
1388             )
1389             return {
1390                 "status": "failed",
1391                 "message": f"cloud compile: could not stage the compile spec: {e}",
1392                 "video_id": video_id,
1393             }
1394
1395         # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
1396         # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
1397         overrides = [f"storage.output_root={out_root}"]
1398         backend_name = (getattr(storage_cfg, "backend", "") or "").strip()

```

```

1399         if backend_name:
1400             overrides.append(f"storage.backend={backend_name}")
1401         spec = CompileJobSpec(
1402             out_uri=out_root,
1403             spec_uri=spec_uri,
1404             video_id=video_id,
1405             output_name=out_name,
1406             config_overrides=tuple(overrides),
1407         )
1408
1409         notify("dispatching stitch (cloud_run)")
1410         logger.info(
1411             "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
1412             video_id,
1413             out_root,
1414             len(kept),
1415         )
1416         try:
1417             result = compute.run_compile(
1418                 spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
1419             )
1420         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1421             logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
1422             return {
1423                 "status": "failed",
1424                 "message": f"cloud compile dispatch error: {e}",
1425                 "video_id": video_id,
1426             }
1427         if result.returncode != 0:
1428             msg = str(
1429                 result.report.get("message")
1430                 or f"cloud compile job failed (returncode {result.returncode})."
1431             )
1432             return {"status": "failed", "message": msg, "video_id": video_id}
1433
1434         download_url = str(
1435             result.report.get("download_url")
1436             or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1437         )
1438         logger.info(
1439             "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
1440             result.execution,
1441             result.outputs_uri,
1442             video_id,
1443         )
1444         notify("finished")
1445         return {
1446             "status": "success",
1447             "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
1448             "download_url": download_url,
1449             "video_id": video_id,

```

14. assistant (2026-07-08T06:15:14.169Z)

I found an unresolved merge-conflict marker at orchestration.py:1306-1311, but I'm on branch `codex/issue-523-sd-reel-plan`, not the merged master. I must review the code as merged in `dac6736`, not local working-tree artifacts. Let me verify the working tree state and read the tests, which are the crux of this test-coverage claim.

15. user (2026-07-08T06:15:17.800Z)

M README.md
M SESSION_HANDOFF.md

```

M backend/config/models.py
M backend/infrastructure/database_jobs.py
M backend/main.py
UU backend/orchestration.py
UU backend/pipeline/stitcher/compiler.py
M config.json
M tests/test_api_endpoints.py
M tests/test_async_jobs.py
M tests/test_compiler.py
M tests/test_watermark.py
?? score_golden_gate.py
?? score_local.py
---DIFF orchestration vs dac6736---
warning: in the working copy of 'backend/orchestration.py', LF will be replaced by CRLF the next
time Git touches it
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 46ffaa4..8b552f8 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -1164,6 +1164,19 @@ def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
     return sc.model_dump() if sc is not None else None

+def _rendition_encode_profile(stitching: Any, rendition: str) -> Optional[Any]:
+    if rendition != "sd":
+        return None
+    renditions = (
+        stitching.get("renditions")
+        if isinstance(stitching, dict)
+        else getattr(stitching, "renditions", None)
+    )
+    if isinstance(renditions, dict):
+        return renditions.get("sd")
+    return getattr(renditions, "sd", None)
+
+def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
+    """A short-lived signed URL for a reel mirrored to the output bucket, so /api/highlights
can
307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the ephemeral
local
@@ -1196,6 +1209,7 @@ def _stitch_reel(
    out_name: str,
    locale: Optional[str],
    notify,
+    rendition: str = "original",
) -> Dict[str, Any]:
    """Stitch + mirror-upload core, shared by the control-plane compile handler AND the L4
worker
(#521). ``time_pairs`` are already RESOLVED (the caller filtered via ``_resolve_compile``),
so this
@@ -1203,24 +1217,41 @@ def _stitch_reel(
    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source, runs
the
trim/watermark/concat, mirrors the reel to ``<output_root>/highlights/<video_id>/<name>``,
and
returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls for.
Returns a
- ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
+ ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg failure).
The
+ optional ``rendition`` selects the output encode profile; ``sd`` scales/transcodes cheaply
and
+ never builds the original-size reel for that job."""

```

16. user (2026-07-08T06:15:18.784Z)

(Bash completed with no output)

17. assistant (2026-07-08T06:15:27.193Z)

The working tree is mid-flight on unrelated issue-523 (SD-rendition) work ? the conflict markers and `\_rendition` code are NOT part of merged #521. I must review the clean `dac6736` state. Let me extract the merged versions of the test files and worker/dispatcher to a scratch dir.

18. user (2026-07-08T06:15:34.093Z)

```
total 316
drwxr-xr-x 1 avidu 197609    0 Jul  8 11:45 .
drwxr-xr-x 1 avidu 197609    0 Jul  8 11:45 ..
-rw-r--r-- 1 avidu 197609  6525 Jul  8 11:45 backend_compute_base.py
-rw-r--r-- 1 avidu 197609 25174 Jul  8 11:45 backend_compute_cloud_run.py
-rw-r--r-- 1 avidu 197609 71951 Jul  8 11:45 backend_orchestration.py
-rw-r--r-- 1 avidu 197609 40742 Jul  8 11:45 backend_worker__main__.py
-rw-r--r-- 1 avidu 197609 49265 Jul  8 11:45 tests_test_api_cloud_dispatch.py
-rw-r--r-- 1 avidu 197609  8939 Jul  8 11:45 tests_test_compile_dispatch.py
-rw-r--r-- 1 avidu 197609 50335 Jul  8 11:45 tests_test_compute_backend.py
-rw-r--r-- 1 avidu 197609 39459 Jul  8 11:45 tests_test_worker.py
```

19. assistant (2026-07-08T06:15:36.646Z)

Now let me read the merged test files that cover the compile dispatcher and worker contract.

20. user (2026-07-08T06:15:37.493Z)

```
1      """Compile/stitch ? L4 worker dispatch (#521).
2
3      Proves the control-plane can OFFLOAD the reel compile/stitch to the Cloud Run L4 worker
by config
4      (`deployment.phase_placement.compile=cloud_run_l4`), mirroring the DETECT dispatch ?
all GPU-free +
5      cloud-free via a FAKE ``CloudRunJobClient`` that simulates the compile worker (reads the
staged
6      compile-spec + writes the reel + done-marker into a local "bucket"). Cardinal invariant:
DEFAULT
7      (compile on the control-plane) still stitches in-process, byte-identically.
8      """
9
10     import json
11     import os
12
13     import pytest
14
15     import backend.compute as compute_pkg
16     import backend.orchestration as orch
17     from backend.compute.cloud_run import CloudRunJobClient
18     from backend.config import AppConfig
19     from backend.infrastructure.database import Database
20     from backend.infrastructure.execution_policy import ExecutionPolicy
21
22     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
23
24
25     class _FakeCompileClient(CloudRunJobClient):
26         """Simulates the Cloud Run compile worker: on dispatch, READ the staged compile-spec
(proving the
27         control-plane resolved + staged it), then write the reel done-marker into the local
bucket exactly
28         where the real worker would, so the dispatcher's bucket-poll resolves."""
```



```

29
30     def __init__(self, *, returncode=0, video_id=None):
31         self.calls: list[list[str]] = []
32         self.spec: dict | None = None
33         self.returncode = returncode
34         self._video_id = video_id
35
36     def run_job(self, job_name, argv, *, region, project=None):
37         self.calls.append(list(argv))
38         spec_uri = argv[argv.index("--spec-uri") + 1]
39         done_uri = argv[argv.index("--done-uri") + 1]
40         with open(spec_uri, encoding="utf-8") as f:
41             self.spec = json.load(f)
42         vid = self._video_id or self.spec["video_id"]
43         out_name = self.spec["output_name"]
44         status = "success" if self.returncode == 0 else "failed"
45         marker = {
46             "video_id": vid,
47             "returncode": self.returncode,
48             "status": status,
49             "download_url": f"/api/highlights/{vid}/{out_name}",
50             "reel_uri": f"gs://bucket/highlights/{vid}/{out_name}",
51             "message": "" if self.returncode == 0 else "stitch blew up",
52         }
53         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
54         with open(done_uri, "w", encoding="utf-8") as f:
55             json.dump(marker, f)
56         return "exec-compile-1"
57
58     def cancel_execution(self, execution): # pragma: no cover - happy path never
59         raise AssertionError("cancel must never fire on the happy path")
60
61
62     def _inject_fake(monkeypatch, fake):
63         real = compute_pkg.get_compute_backend
64         monkeypatch.setattr(
65             compute_pkg,
66             "get_compute_backend",
67             lambda config, **kw: real(config, run_client=fake, policy=_FAST, token="tok",
68 **kw),
69         )
70
71     @pytest.fixture
72     def compile_env(tmp_path, monkeypatch):
73         """A control-plane DB with a video + 3 play segments, wired for cloud with compile
74         placed on the L4 and a LOCAL bucket (so the fake worker's spec/marker are plain file
75         reads/writes)."""
76         db = Database(str(tmp_path / "t.db"))
77         bucket = tmp_path / "bucket"
78         bucket.mkdir()
79         video_id = "vidCompile"
80         src = tmp_path / "source.mp4"
81         src.write_bytes(b"SRC")
82         db.add_video(video_id, str(src), "processed", [])
83         sid1 = db.add_play_segment(video_id, 2.0, 6.0, metadata={"shot_count": 5})
84         sid2 = db.add_play_segment(video_id, 10.0, 14.0, metadata={"shot_count": 3})
85         sid3 = db.add_play_segment(video_id, 20.0, 22.0, metadata={"shot_count": 4})
86         cfg = AppConfig.model_validate(
87             {
88                 "deployment": {
89                     "compute_target": "cloud_run",
90                     "phase_placement": {"compile": "cloud_run_l4"},

```

```

90         },
91         "storage": {"backend": "local", "output_root": str(bucket)},
92     }
93 )
94 return db, cfg, video_id, [sid1, sid2, sid3], bucket, monkeypatch
95
96
97 def _job(video_id, segment_ids, name="reel.mp4", locale=None):
98     return {
99         "id": "cj1",
100         "video_id": video_id,
101         "kind": "compile",
102         "params": json.dumps(
103             {"segment_ids": segment_ids, "output_filename": name, "locale": locale}
104         ),
105     }
106
107
108 def test_compile_dispatches_to_l4_and_returns_reel(compile_env):
109     db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
110     fake = _FakeCompileClient()
111     _inject_fake(monkeypatch, fake)
112
113     out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "myreel.mp4"))
114
115     assert out["status"] == "success"
116     assert out["video_id"] == video_id
117     assert out["download_url"] == f"/api/highlights/{video_id}/myreel.mp4"
118     assert out["compute"]["path"] == "cloud_run"
119     assert out["compute"]["execution"] == "exec-compile-1"
120     assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
121
122
123 def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
124     """The staged compile-spec carries the RESOLVED time-pairs (padding-agnostic
125     start/end from the DB)
126     and the L4 encoder (deployment.cloud_stitch_encoder = h264_nvenc by default) ? so
127     the worker
128     hardware-encodes and needs no control-plane DB."""
129     db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
130     fake = _FakeCompileClient()
131     _inject_fake(monkeypatch, fake)
132
133     orch._execute_compile(cfg, db, _job(video_id, seg_ids))
134
135     assert fake.spec is not None
136     assert fake.spec["video_id"] == video_id
137     assert fake.spec["video_uri"].endswith("source.mp4")
138     # 3 resolved (start, end) pairs from the DB rows
139     assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
140     # the L4 stitch uses NVENC (forwarded from deployment.cloud_stitch_encoder)
141     assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
142     # the argv is a `compile` subcommand carrying the spec + done marker
143     argv = fake.calls[0]
144     assert argv[0] == "compile"
145     assert "--spec-uri" in argv and "--done-uri" in argv
146     joined = " ".join(argv)
147     assert "storage.output_root=" in joined # bucket forwarded so the worker mirrors
148     the reel
149
150
151 def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
152     """The worker Job has no DEPLOYMENT_PROFILE, so the dispatch forwards the bucket
153     coordinates
154     (storage.output_root/backend) ? else the worker can't read the spec or mirror the

```

```

reel."""
151     db, cfg, video_id, seg_ids, bucket, monkeypatch = compile_env
152     fake = _FakeCompileClient()
153     _inject_fake(monkeypatch, fake)
154     orch._execute_compile(cfg, db, _job(video_id, seg_ids))
155     joined = " ".join(fake.calls[0])
156     assert f"storage.output_root={bucket}" in joined
157     assert "storage.backend=local" in joined
158
159
160     def test_compile_cloud_failure_returns_failed(compile_env):
161         """A worker stitch failure (rc != 0) surfaces as a failed job with the worker's
message ? the SPA
162         gets a clean verdict, not a crash."""
163         db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
164         fake = _FakeCompileClient(returncode=3)
165         _inject_fake(monkeypatch, fake)
166
167         out = orch._execute_compile(cfg, db, _job(video_id, seg_ids))
168         assert out["status"] == "failed"
169         assert "stitch blew up" in out["message"]
170         assert out["video_id"] == video_id
171
172
173     def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
174         """DEFAULT (compile on the control-plane): _execute_compile stitches in-process via
_stitch_reel
175         and NEVER dispatches to Cloud Run. The stitch core is stubbed so no real ffmpeg
runs."""
176         db, cfg, video_id, seg_ids, _bucket, _mp = compile_env
177         # flip compile back to the control-plane (the default) ? everything else stays
cloud-wired
178         cfg.deployment.phase_placement.compile = "control_plane"
179         fake = _FakeCompileClient()
180         _inject_fake(monkeypatch, fake)
181         seen = {}
182
183         def _fake_stitch(c, stitching, vid, source, pairs, name, locale, notify):
184             seen["pairs"] = pairs
185             seen["source"] = source
186             return {"status": "success", "video_id": vid, "download_url":
f"/api/highlights/{vid}/{name}"}
187
188         monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
189
190         out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "local.mp4"))
191         assert out["status"] == "success"
192         assert fake.calls == [] # NEVER dispatched ? the control-plane stitched in-process
193         assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0), (20.0, 22.0)]
194         assert seen["source"].endswith("source.mp4")
195
196
197     def test_compile_missing_video_fails_before_dispatch(compile_env):
198         """A compile for an unknown video_id fails at resolve (404) before any dispatch."""
199         db, cfg, _video_id, _seg_ids, _bucket, monkeypatch = compile_env
200         fake = _FakeCompileClient()
201         _inject_fake(monkeypatch, fake)
202         out = orch._execute_compile(cfg, db, _job("nope", [1, 2]))
203         assert out["status"] == "failed"
204         assert fake.calls == [] # never dispatched ? resolve failed first
205

```

21. assistant (2026-07-08T06:15:54.161Z)

`test\_compile\_dispatch.py` exercises the producer (`\_execute\_compile` ?

`\_dispatch\_compile\_remote` ? `run\_compile` ? argv), but the `\_FakeCompileClient` re-implements the consumer by hand (reads `spec["video\_id"]`/`spec["output\_name"]` directly, never calls `build\_parser`/`cmd\_compile`/`\_read\_compile\_spec`). This is central to the claim. Let me check the worker-side and compute-backend tests.

22. user (2026-07-08T06:15:58.593Z)

```
84:     args = wmain.build_parser().parse_args(
97:     assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
104:     assert wmain.build_parser().parse_args(base).skip_validation is False
106:     assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
253:     args = wmain.build_parser().parse_args(
813:     args = wmain.build_parser().parse_args(
949:# ----- compile subcommand (#521 ? L4
stitch)
952:def test_parser_compile_accepts_spec_and_done_uris():
953:     args = wmain.build_parser().parse_args(
955:         "compile",
957:         "gs://b/_compile/tok/spec.json",
961:         "gs://b/_compile/tok.done",
966:     assert args.cmd == "compile"
967:     assert args.spec_uri == "gs://b/_compile/tok/spec.json"
968:     assert args.done_uri == "gs://b/_compile/tok.done"
972:def _write_compile_spec(bucket, video_id, source, name, encoder="h264_nvenc"):
975:     "video_uri": str(source),
976:     "output_name": name,
978:     "time_pairs": [[2.0, 6.0], [10.0, 14.0]],
981:     spec_dir = os.path.join(str(bucket), "_compile", "tok")
983:     spec_uri = os.path.join(spec_dir, "spec.json")
984:     with open(spec_uri, "w", encoding="utf-8") as f:
986:         return spec_uri
989:def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
990:     """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core (VideoCompiler
stubbed so
991:     no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
bucket-poll."""
998:     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
999:     done = str(bucket / "_compile" / "tok.done")
1003:     def _fake_stitch(cfg, stitching, video_id, source_ref, time_pairs, out_name, locale,
notify):
1005:         seen["pairs"] = time_pairs
1006:         seen["source"] = source_ref
1019:     rc = wmain.cmd_compile(
1020:         wmain.build_parser().parse_args(
1022:             "compile",
1024:             spec_uri,
1048:def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
1057:     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
1058:     done = str(bucket / "_compile" / "tok.done")
1067:     rc = wmain.cmd_compile(
1068:         wmain.build_parser().parse_args(
1070:             "compile",
1072:             spec_uri,
1089:def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
1097:     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
1104:     rc = wmain.cmd_compile(
1105:         wmain.build_parser().parse_args(
1107:             "compile",
1109:             spec_uri,
1120:     assert not (bucket / "_compile" / "tok.done").exists()
```

23. user (2026-07-08T06:16:01.034Z)

```
40:     def run_job(self, job_name, argv, *, region, project=None):
```

```

42:         {"job": job_name, "argv": list(argv), "region": region, "project": project}
44:         out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
45:         done_uri = argv[argv.index("--done-uri") + 1]
84:     def run_job(self, job_name, argv, *, region, project=None):
86:         {"job": job_name, "argv": list(argv), "region": region, "project": project}
105:     def run_job(self, job_name, argv, *, region, project=None):
107:         {"job": job_name, "argv": list(argv), "region": region, "project": project}
131:     def run_job(self, job_name, argv, *, region, project=None):
133:         {"job": job_name, "argv": list(argv), "region": region, "project": project}
220:         video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"
236: def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
245:     video_uri="gs://b/v.mp4",
252:     argv = client.calls[0]["argv"]
253:     assert argv[0] == "infer"
254:     assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
255:     assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
260:     assert "--skip-validation" not in argv
262:     joined = " ".join(argv)
272: def test_dispatch_argv_carries_skip_validation_when_set(tmp_path):
283:     video_uri="gs://b/v.mp4",
288:     assert "--skip-validation" in client.calls[0]["argv"]
296:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
299:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
301:     done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
302:     done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
367:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
394:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
416:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
453:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
483:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
510:     video_uri="gs://b/v.mp4",
535:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
555:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
580:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
634:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
745:     wmain.build_parser().parse_args(
760:     assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
803:     wmain.build_parser().parse_args(
830:     assert report["video_uri"] == str(video)
868:     wmain.build_parser().parse_args(
922:     wmain.build_parser().parse_args(
979:     wmain.build_parser().parse_args(
1098:     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk")),
1105: # ----- compile dispatch (#521 ? L4
stitch)
1110:     bucket exactly where the real ``cmd_compile`` would, so ``run_compile``'s bucket-poll
resolves. """
1116:     def run_job(self, job_name, argv, *, region, project=None):
1117:         self.calls.append({"job": job_name, "argv": list(argv)})
1118:         done_uri = argv[argv.index("--done-uri") + 1]
1131:         return "exec-compile"
1137: def test_run_compile_dispatches_and_returns_marker(tmp_path):
1138:     """run_compile dispatches a `compile` subcommand (spec + done marker), bucket-polls,
and returns
1140:     from backend.compute.base import CompileJobSpec
1147:     result = backend.run_compile(
1148:         CompileJobSpec(
1150:             spec_uri=f"{out}/_compile/tok/spec.json",
1152:             output_name="reel.mp4",
1159:             argv = client.calls[0]["argv"]
1160:             assert argv[0] == "compile"
1161:             assert argv[argv.index("--spec-uri") + 1] == f"{out}/_compile/tok/spec.json"
1162:             assert "--done-uri" in argv
1163:             # the CP-forwarded storage override rides as --set (compile carries NO infer container
overrides)

```

```

1164:     assert "storage.output_root=" + out in " ".join(argv)
1165:     assert "deployment.compute_target=local_gpu" not in " ".join(argv)
1168: def test_run_compile_surfaces_nonzero_returncode(tmp_path):
1171:     from backend.compute.base import CompileJobSpec
1178:     result = backend.run_compile(
1179:         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",
output_name="r.mp4")
1184: def test_compile_marker_prefix_distinct_from_infer(tmp_path):
1185:     """The compile marker lives under _compile/ (not infer's _dispatch/) so a compile + a
detection
1187:     from backend.compute.base import CompileJobSpec
1194:     backend.run_compile(
1195:         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",
output_name="r.mp4")
1197:     done_uri = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
1198:     assert "/_compile/" in done_uri and "/_dispatch/" not in done_uri
1201: def test_run_compile_default_backend_raises_not_implemented():
1203:     CloudRunCompute overrides run_compile) ? never a silent no-op."""
1204:     from backend.compute.base import CompileJobSpec, ComputeBackend, ComputeResult
1211:     _InProc().run_compile(
1212:         CompileJobSpec(out_uri="x", spec_uri="y", video_id="v", output_name="r.mp4")

```

24. assistant (2026-07-08T06:16:32.494Z)

Let me read the exact worker-side test bodies and the compute-backend fake compile client to confirm neither round-trips through the real counterpart.

25. user (2026-07-08T06:16:35.878Z)

```

1105 # ----- compile dispatch (#521 ? L4
stitch)
1106
1107
1108 class _FakeCompileClient(CloudRunJobClient):
1109     """Simulates the Cloud Run COMPILER worker: on dispatch, write a reel done-marker
into the local
1110     bucket exactly where the real ``cmd_compile`` would, so ``run_compile``'s bucket-
poll resolves."""
1111
1112     def __init__(self, *, video_id="vidReel", returncode=0):
1113         self.calls: list[dict] = []
1114         self.video_id, self.returncode = video_id, returncode
1115
1116     def run_job(self, job_name, argv, *, region, project=None):
1117         self.calls.append({"job": job_name, "argv": list(argv)})
1118         done_uri = argv[argv.index("--done-uri") + 1]
1119         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
1120         with open(done_uri, "w", encoding="utf-8") as f:
1121             json.dump(
1122                 {
1123                     "video_id": self.video_id,
1124                     "returncode": self.returncode,
1125                     "status": "success" if self.returncode == 0 else "failed",
1126                     "download_url": f"/api/highlights/{self.video_id}/reel.mp4",
1127                     "reel_uri": f"gs://b/highlights/{self.video_id}/reel.mp4",
1128                 },
1129                 f,
1130             )
1131         return "exec-compile"
1132
1133     def cancel_execution(self, execution): # pragma: no cover - happy path never
cancels
1134         raise AssertionError("cancel must never fire on the happy path")
1135
1136

```

```

1137 def test_run_compile_dispatches_and_returns_marker(tmp_path):
1138     """run_compile dispatches a `compile` subcommand (spec + done marker), bucket-polls,
and returns
1139     the worker's rc + download_url from the marker."""
1140     from backend.compute.base import CompileJobSpec
1141
1142     client = _FakeCompileClient(video_id="vidReel")
1143     backend = CloudRunCompute(
1144         run_client=client, job_name="rally-worker", region="r", policy=_FAST,
token="tok"
1145     )
1146     out = str(tmp_path / "bucket")
1147     result = backend.run_compile(
1148         CompileJobSpec(
1149             out_uri=out,
1150             spec_uri=f"{out}/_compile/tok/spec.json",
1151             video_id="vidReel",
1152             output_name="reel.mp4",
1153             config_overrides=("storage.output_root=" + out,),
1154         )
1155     )
1156     assert result.returncode == 0
1157     assert result.video_id == "vidReel"
1158     assert result.report["download_url"] == "/api/highlights/vidReel/reel.mp4"
1159     argv = client.calls[0]["argv"]
1160     assert argv[0] == "compile"
1161     assert argv[argv.index("--spec-uri") + 1] == f"{out}/_compile/tok/spec.json"
1162     assert "--done-uri" in argv
1163     # the CP-forwarded storage override rides as --set (compile carries NO infer
container overrides)
1164     assert "storage.output_root=" + out in " ".join(argv)
1165     assert "deployment.compute_target=local_gpu" not in " ".join(argv)
1166
1167
1168 def test_run_compile_surfaces_nonzero_returncode(tmp_path):
1169     """A worker stitch failure (rc != 0) is returned as a non-zero ComputeResult (the
dispatcher maps
1170     it to a failed job) ? never a false success."""
1171     from backend.compute.base import CompileJobSpec
1172
1173     client = _FakeCompileClient(returncode=3)
1174     backend = CloudRunCompute(
1175         run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
1176     )
1177     out = str(tmp_path / "bk")
1178     result = backend.run_compile(
1179         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",
output_name="r.mp4")
1180     )
1181     assert result.returncode == 3
1182
1183
1184 def test_compile_marker_prefix_distinct_from_infer(tmp_path):
1185     """The compile marker lives under _compile/ (not infer's _dispatch/) so a compile +
a detection
1186     for the same bucket never collide on a marker."""
1187     from backend.compute.base import CompileJobSpec
1188
1189     client = _FakeCompileClient()
1190     backend = CloudRunCompute(
1191         run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
1192     )
1193     out = str(tmp_path / "bk")
1194     backend.run_compile(
1195         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",

```

```

output_name="r.mp4")
1196     )
1197     done_uri = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
1198     assert "/_compile/" in done_uri and "/_dispatch/" not in done_uri
1199
1200
1201 def test_run_compile_default_backend_raises_not_implemented():
1202     """The ABC default: a backend that can't offload a stitch raises NotImplementedError
(only
1203     CloudRunCompute overrides run_compile) ? never a silent no-op."""
1204     from backend.compute.base import CompileJobSpec, ComputeBackend, ComputeResult
1205
1206     class _InProc(ComputeBackend):
1207         def run_infer(self, spec, *, on_wait=None):
1208             return ComputeResult(returncode=0)
1209
1210     with pytest.raises(NotImplementedError):
1211         _InProc().run_compile(
1212             CompileJobSpec(out_uri="x", spec_uri="y", video_id="v", output_name="r.mp4")
1213         )
1214

```

26. user (2026-07-08T06:16:36.775Z)

```

949     # ----- compile subcommand (#521 ?
L4 stitch)
950
951
952 def test_parser_compile_accepts_spec_and_done_uris():
953     args = wmain.build_parser().parse_args(
954         [
955             "compile",
956             "--spec-uri",
957             "gs://b/_compile/tok/spec.json",
958             "--out-uri",
959             "gs://b",
960             "--done-uri",
961             "gs://b/_compile/tok.done",
962             "--set",
963             "storage.output_root=gs://b",
964         ]
965     )
966     assert args.cmd == "compile"
967     assert args.spec_uri == "gs://b/_compile/tok/spec.json"
968     assert args.done_uri == "gs://b/_compile/tok.done"
969     assert args.set == ["storage.output_root=gs://b"]
970
971
972 def _write_compile_spec(bucket, video_id, source, name, encoder="h264_nvenc"):
973     spec = {
974         "video_id": video_id,
975         "video_uri": str(source),
976         "output_name": name,
977         "locale": None,
978         "time_pairs": [[2.0, 6.0], [10.0, 14.0]],
979         "stitching": {"encoder": encoder, "watermark": {"enabled": False}},
980     }
981     spec_dir = os.path.join(str(bucket), "_compile", "tok")
982     os.makedirs(spec_dir, exist_ok=True)
983     spec_uri = os.path.join(spec_dir, "spec.json")
984     with open(spec_uri, "w", encoding="utf-8") as f:
985         json.dump(spec, f)
986     return spec_uri
987
988

```



```

989     def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
990         """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core
(VideoCompiler stubbed so
991         no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
bucket-poll."""
992         import backend.orchestration as orch
993
994         bucket = tmp_path / "bucket"
995         bucket.mkdir()
996         source = tmp_path / "src.mp4"
997         source.write_bytes(b"SRC")
998         spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
999         done = str(bucket / "_compile" / "tok.done")
1000
1001         seen = {}
1002
1003     def _fake_stitch(cfg, stitching, video_id, source_ref, time_pairs, out_name, locale,
notify):
1004         seen["encoder"] = stitching.encoder
1005         seen["pairs"] = time_pairs
1006         seen["source"] = source_ref
1007         notify("stitching")
1008         return {
1009             "status": "success",
1010             "video_id": video_id,
1011             "download_url": f"/api/highlights/{video_id}/{out_name}",
1012             "reel_uri": f"gs://b/highlights/{video_id}/{out_name}",
1013         }
1014
1015     monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
1016
1017     cfg = tmp_path / "c.json"
1018     cfg.write_text('{"storage": {"backend": "local"}}')
1019     rc = wmain.cmd_compile(
1020         wmain.build_parser().parse_args(
1021             [
1022                 "compile",
1023                 "--spec-uri",
1024                 spec_uri,
1025                 "--out-uri",
1026                 str(bucket),
1027                 "--done-uri",
1028                 done,
1029                 "--config",
1030                 str(cfg),
1031                 "--scratch",
1032                 str(tmp_path / "s"),
1033                 "--set",
1034                 f"storage.output_root={bucket}",
1035             ]
1036         )
1037     )
1038     assert rc == 0
1039     # the shared stitch core got the resolved pairs + the NVENC encoder from the spec
1040     assert seen["encoder"] == "h264_nvenc"
1041     assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0)]
1042     assert seen["source"] == str(source)
1043     marker = json.loads(open(done, encoding="utf-8").read())
1044     assert marker["returncode"] == 0 and marker["status"] == "success"
1045     assert marker["download_url"] == "/api/highlights/vidReel/reel.mp4"
1046
1047
1048     def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
1049         """A stitch failure surfaces as rc 3 + a failed marker (rc != 0) so the dispatcher's
poll

```

```

1050     terminates and the SPA sees a clean failure."""
1051     import backend.orchestration as orch
1052
1053     bucket = tmp_path / "bucket"
1054     bucket.mkdir()
1055     source = tmp_path / "src.mp4"
1056     source.write_bytes(b"SRC")
1057     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
1058     done = str(bucket / "_compile" / "tok.done")
1059
1060     monkeypatch.setattr(
1061         orch,
1062         "_stitch_reel",
1063         lambda *a, **k: {"status": "failed", "message": "ffmpeg blew up", "video_id":
"vidReel"},
1064     )
1065     cfg = tmp_path / "c.json"
1066     cfg.write_text('{"storage": {"backend": "local"}}')
1067     rc = wmain.cmd_compile(
1068         wmain.build_parser().parse_args(
1069             [
1070                 "compile",
1071                 "--spec-uri",
1072                 spec_uri,
1073                 "--out-uri",
1074                 str(bucket),
1075                 "--done-uri",
1076                 done,
1077                 "--config",
1078                 str(cfg),

```

27. assistant (2026-07-08T06:17:40.746Z)

I now have the full picture of both sides of the contract and all four tests. Let me verify one remaining detail ? that the consumer reads spec keys defensively with `.get()` (relevant to the claim's "KeyErrors" mechanism), which I already saw at worker `__main__.py:488-496`, and confirm the parser accepts every flag the producer emits.

Let me compile my analysis.

****Contract consistency check (merged `dac6736`):****

Producer argv ? ``cloud_run.py:323-335``: ``["compile", "--out-uri", ..., "--spec-uri", ..., "--done-uri", ..., "--set", kv...]`
Consumer parser ? ``worker/__main__.py:865-900``: compile subparser defines ``--spec-uri`` (required), ``--out-uri``, ``--done-uri``, ``--config``, ``--scratch``, ``--set``. Every producer flag is accepted. No drift.

Producer spec ? ``orchestration.py:1366-1375``: ``{video_id, video_uri, output_name, locale, time_pairs, stitching}``
Consumer reads ? ``worker/__main__.py:489-496``: ``spec.get("video_id")``, ``spec.get("output_name")``, ``spec.get("video_uri")``, ``spec.get("locale")``, ``spec.get("time_pairs")``, ``spec.get("stitching")``. Every key matches, and all reads are defensive `.get(...)` with ``or ""`` or `[]`` or `{}`` defaults.

****Test structure (matches the claim's description):****

- ``test_run_compile_dispatches_and_returns_marker`` ? real ``run_compile`` emits argv; fake client indexes argv positionally, never calls ``build_parser``.
- ``test_parser_compile_accepts_spec_and_done_uris`` ? hand-written argv fed to ``build_parser``.
- ``test_compile_dispatch.py`` ? real ``dispatch_compile_remote`` stages spec; fake worker reads only ``spec["video_id"]``/``spec["output_name"]``, never calls ``cmd_compile``/``_read_compile_spec``.
- ``test_cmd_compile_stitches_and_writes_marker`` ? real ``cmd_compile`` reads a hand-built ``_write_compile_spec``.

So the claim's factual description is accurate: no single test binds the **real** producer's output to the **real** consumer's input.

28. user (2026-07-08T06:17:53.663Z)

Structured output provided successfully